

```
local-path-storage Active 2m58s
```

And let's check the various logs that are running:

```
> kubectl get pods -A
NAMESPACE          NAME
READY STATUS RESTARTS AGE
kube-system        coredns-558bd4d5db-7pzbj
1/1 Running 0 3m25s
kube-system        coredns-558bd4d5db-vgf9l
1/1 Running 0 3m25s
kube-system        etcd-kind-control-plane
1/1 Running 0 3m28s
kube-system        kindnet-jq54g
1/1 Running 0 3m25s
kube-system        kube-apiserver-kind-control-plane
1/1 Running 0 3m28s
kube-system        kube-controller-manager-kind-control-plane
1/1 Running 0 3m28s
kube-system        kube-proxy-bkxql
1/1 Running 0 3m25s
kube-system        kube-scheduler-kind-control-plane
1/1 Running 0 3m28s
local-path-storage local-path-provisioner-547f784dff-2vkf4
1/1 Running 0 3m25s
```

Loading a Docker image: To load a Docker image, give the following command:

```
kind load docker-image image_1 image_2
```

Note that once you load the image, you still have to create the Deployment k8s resource, which will typically reference the loaded image and then do kubectl apply of the resource.

NGINX Ingress controller: Now let's deploy Ingress NGINX to our cluster:

```
> kubectl apply -f https://raw.githubusercontent.com/
kubernetes/ingress-nginx/main/deploy/static/provider/kind/
deploy.yaml
namespace/ingress-nginx created
serviceaccount/ingress-nginx created
serviceaccount/ingress-nginx-admission created
role.rbac.authorization.k8s.io/ingress-nginx created
role.rbac.authorization.k8s.io/ingress-nginx-admission created
clusterrole.rbac.authorization.k8s.io/ingress-nginx created
clusterrole.rbac.authorization.k8s.io/ingress-nginx-admission
created
rolebinding.rbac.authorization.k8s.io/ingress-nginx created
rolebinding.rbac.authorization.k8s.io/ingress-nginx-admission
created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx
```

```
created
clusterrolebinding.rbac.authorization.k8s.io/ingress-nginx-
admission created
configmap/ingress-nginx-controller created
service/ingress-nginx-controller created
service/ingress-nginx-controller-admission created
deployment.apps/ingress-nginx-controller created
job.batch/ingress-nginx-admission-create created
job.batch/ingress-nginx-admission-patch created
ingressclass.networking.k8s.io/nginx created
validatingwebhookconfiguration.admissionregistration.k8s.io/
ingress-nginx-admission created
```

Next, let's inspect our NGINX pods:

```
> kubectl get pods -n ingress-nginx
NAME                                READY STATUS RESTARTS AGE
ingress-nginx-admission-create-hrmn5 0/1 Completed
0 47s
ingress-nginx-admission-patch-7ds99 0/1 Completed
1 47s
ingress-nginx-controller-674cb6ff57-tmjbc 0/1
ContainerCreating 0 47s
```

Let's check the service of the NGINX controller:

```
> kubectl get services -n ingress-nginx
NAME                                TYPE CLUSTER-IP
EXTERNAL-IP PORT(S) AGE
ingress-nginx-controller            NodePort 10.96.65.28
<none> 80:32544/TCP, 443:30304/TCP 116s
ingress-nginx-controller-admission ClusterIP 10.96.207.79
<none> 443/TCP 116s
```

You can see the NGINX controller is listening on both port 80 and 443. Now let's check what our Ingress class is:

```
> kubectl get ingressclass -A
NAME CONTROLLER PARAMETERS AGE
nginx k8s.io/ingress-nginx <none> 8m26s
```

Deploying an application

Let's deploy the <https://httpbin.org/> application inside our Kind cluster. I have the manifest for the application in the app_k8s repository: https://github.com/psibi/app_k8s.

Rendering the resource for the application will result in this:

```
> kustomize build overlays/app/nginx
apiVersion: v1
kind: Namespace
metadata:
```